

Darshan Ease: Temple Darshan Ticket Booking System

Project Description:

Darshan Ease brings a full-stack web platform to temple darshan ticket booking, replacing long offline queues with a fast, secure, and personalized online booking experience. The platform includes a Temple / Darshan Browsing module, which lists available temples and darshan slots with details such as timing and description; a Ticket Booking module, which lets a devotee select a slot and create a confirmed booking in seconds; and a Booking History module, which gives every user a running record of their current and past darshan tickets.

Built with a React.js frontend and a Node.js / Express.js backend following the MVC architectural pattern, Darshan Ease uses MongoDB (via Mongoose) to persist users, temple/product records, and orders. Authentication is handled with JSON Web Tokens (JWT), so every booking and profile action is tied to a securely verified user. With environment-based configuration via .env and centralized error handling on every route, Darshan Ease empowers devotees to book darshan tickets with confidence, and gives temple administrators a straightforward way to manage temples, slots, and orders.

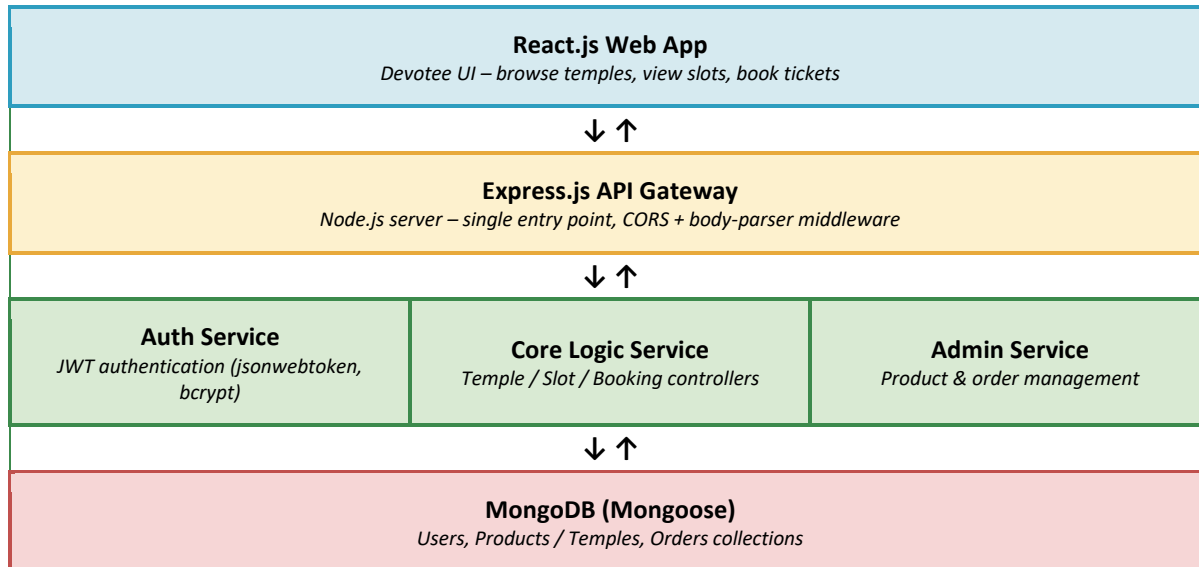
Scenarios

Scenario 1: A devotee planning a weekend family trip registers on Darshan Ease, browses the list of temples, and selects a temple with an upcoming auspicious day. She views the available darshan slots, picks a convenient morning slot, and books it. Within seconds, she receives an electronic ticket confirmation and can see the booking in her profile's order history — no counter visit or phone call required.

Scenario 2: An NRI devotee visiting India for a short trip logs into Darshan Ease from abroad ahead of time, checks slot availability for the exact dates he'll be in the city, and books a slot in advance — avoiding the risk of travelling without a confirmed darshan slot.

Scenario 3: A temple administrator logs into the admin side of Darshan Ease during a festival week, adds a new set of special darshan slots for the occasion, and monitors incoming orders in real time — updating or managing bookings as needed without any manual, paper-based process.

Technical Architecture:



Description: Darshan Ease uses a modular three-tier architecture to deliver a responsive booking experience. The frontend is a React.js single-page application, while the Express.js backend orchestrates request routing, input validation, and business logic. It communicates with a MongoDB database via Mongoose for all persistent data, with JWT-based authentication securing every protected route.

(Note: Darshan Ease uses a database – MongoDB – so the ER relationships are: a User can place many Orders; each Order references one Product/Temple; each Product/Temple can appear in many Orders.)

Pre-requisites:

- **Express.js Framework Knowledge:** <https://expressjs.com/>
- **MongoDB & Mongoose Familiarity:** <https://mongoosejs.com/docs/>
- **HTML, CSS, and React Skills:** <https://react.dev/>
- **JavaScript Programming Proficiency:** <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- **Version Control with Git:** <https://git-scm.com/doc>
- **Node.js Development Environment Setup:** <https://nodejs.org/en/download>
- **JWT Authentication Concepts:** <https://jwt.io/introduction>

Project Workflow:

Milestone 1: Backend Setup and Architecture

- **Activity 1.1:** Initialize the Node.js project and install Express, Mongoose, jsonwebtoken, bcrypt, cors, and body-parser.
- **Activity 1.2:** Set up a MongoDB connection (local or MongoDB Atlas) via a dedicated config/db.js module.
- **Activity 1.3:** Define the application architecture, detailing interactions between the React frontend, the Express API layer, and MongoDB.
- **Activity 1.4:** Set up the development environment, structuring the project into client/ and server/ folders with routes, controllers, models, and middleware.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: user Registration & Login, Temple/Product Browsing, Darshan Slot Booking, and Booking History.
- **Activity 2.2:** Implement the Express backend to manage routing and request validation, ensuring smooth interaction between frontend and database.

Milestone 3: Server (index.js) Development

- **Activity 3.1:** Write the main application logic in index.js, wiring up middleware and mounting each feature's routes.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using React, ensuring a responsive and intuitive layout across Register, Login, Products, Profile, and Orders pages.
- **Activity 4.2:** Create dynamic components that fetch data from the backend API and render it for the user.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring environment variables and installing all dependencies for both client and server.
- **Activity 5.2:** Run and verify the client and server locally, confirming the full registration-to-booking flow works end-to-end.

Milestone 6: Conclusion

Milestone 1: Backend Setup and Architecture

In this milestone, the focus is on setting up the Node.js/Express backend project and defining how the frontend, backend, and database will interact for Darshan Ease.

Activity 1.1: Initialize the Backend Project

Before writing any routes, the server project needs to be initialized and its core dependencies installed.

- **Step 1 — Initialize the project:** Inside the server folder, run `npm init -y` to create a `package.json` file.
- **Step 2 — Install core dependencies:** Install Express, Mongoose, JWT, and related middleware packages.

```
server> npm install express mongoose jsonwebtoken bcrypt cors body-parser dotenv
server> npm install --save-dev nodemon
```

- **Step 3 — Configure the API Key / secrets:** Create a `.env` file in the server root and add the MongoDB connection string and port:

```
MONGO_URI=mongodb://localhost:27017/darshanease
PORT=8000
JWT_SECRET=your_jwt_secret_here
```

- **Important — Never Commit Your .env File:** Add `.env` to `.gitignore` to prevent secrets from being pushed to a public repository:

```
.env
```

Activity 1.2: Configure the MongoDB Connection

A dedicated `config/db.js` module handles connecting to MongoDB using Mongoose, keeping database setup separate from the rest of the application logic.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application, including the React frontend, Express backend, and MongoDB database (see Technical Architecture section).
- **Detail Frontend Functionality:** Outline how users interact with the app — registering, logging in, browsing temples, selecting a slot, and booking a ticket.
- **Outline Backend Responsibilities:** Specify how the Express backend handles incoming requests to `/api/auth`, `/api/products`, and `/api/orders`, validates input, and communicates with MongoDB via Mongoose models.
- **Describe Security Integration Points:** Define how `authMiddleware.js` verifies the JWT on protected routes and attaches the authenticated user to the request before it reaches the controller.

Activity 1.4: Set Up the Development Environment

- **Set Up Application Structure:** Create the project directory structure with folders for routes, controllers, models, middleware, and config, plus the main entry file (index.js):

```
server/  
  config/  
    db.js  
  models/  
    User.js  
    Product.js  
    Order.js  
  routes/  
    authRoutes.js  
    productRoutes.js  
    orderRoutes.js  
  controllers/  
    authController.js  
    productController.js  
    orderController.js  
  middleware/  
    authMiddleware.js  
  .env  
  index.js  
  package.json
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content & Booking Features

Implement the core features — Registration, Login, Temple/Product Browsing, and Ticket Booking — each backed by its own route, controller, and Mongoose model.

- **Registration & Login:** Accepts a name, email, and password; hashes the password with bcrypt on registration and issues a signed JWT on successful login.
- **Temple / Product Browsing:** Returns the list of temples/darshan products, including name, description, and price, for the frontend to render.
- **Ticket Booking:** Accepts a product/slot selection from an authenticated user and creates a new Order document linking the user and the product.

Activity 2.2: Implement the Express Backend

- **Define Routes:** Set up route files in routes/ for authentication, products, and orders, each mapped to its controller.
- **Process User Input:** Read JSON request bodies (e.g. name, email, password, or product_id) using Express's built-in JSON parsing.
- **Validate Input:** Return a 400 error with a clear message if required fields are missing or invalid before touching the database.
- **Protect Routes:** Apply authMiddleware to routes like /api/orders and /api/profile so only authenticated users can access them.

Milestone 3: Server (index.js) Development

Milestone 3 focuses on the main entry point of the backend, index.js, which wires together the Express app, middleware, and all feature routes, plus the authentication middleware and order routes that protect and power the booking flow.

Activity 3.1: Writing the Main Application Logic in index.js

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
const connectDB = require("../config/db");
const authRoutes = require("../routes/authRoutes");

const app = express();

connectDB();

app.use(cors());
app.use(bodyParser.json());

app.use("/api/auth", authRoutes);

const PORT = process.env.PORT;

app.get("/", (req, res) => {
  res.send("DarshanEase Backend Running Successfully");
});

app.get("/test", (req, res) => {
  res.send("Backend Working Fine");
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Define the Authentication Middleware

authMiddleware.js verifies the JWT sent in the Authorization header and attaches the decoded user to the request, protecting routes like order creation and profile access.

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader) {
      return res.status(401).json({
        message: "Access Denied. No Token Provided",
      });
    }
  }
}
```

```

const token = authHeader.split(" ")[1];

const verified = jwt.verify(token, "mysecretkey");

req.user = verified;

next();
} catch (error) {
  res.status(401).json({
    message: "Invalid Token",
  });
}
};

module.exports = authMiddleware;

```

Define the Order Routes

orderRoutes.js exposes full CRUD endpoints for bookings, each mapped to its controller function.

```

const express = require("express");
const router = express.Router();

const {
  createOrder,
  getOrders,
  getOrderById,
  updateOrder,
  deleteOrder,
} = require("../controllers/orderController");

router.post("/", createOrder);
router.get("/", getOrders);
router.get("/:id", getOrderById);
router.put("/:id", updateOrder);
router.delete("/:id", deleteOrder);

module.exports = router;

```


Milestone 4: Frontend Development

Milestone 4 focuses on building a clean, responsive React interface for Darshan Ease, covering registration, login, temple/product browsing, and booking, with each page fetching data from the Express backend.

Activity 4.1: Designing and Developing the User Interface

- **Set Up the Base App Structure:** Create the React app with routing configured for Home, Register, Login, Products, Profile, and Orders pages.
- **Create a Shared API Service:** Centralize backend calls in a single Axios instance (services/api.js) so every component reuses the same base URL and headers.
- **Design Components for Each Page:** Build a Register form, a Login form, a Products list, and an Orders/Booking history view, each as its own component.

Activity 4.2: Creating Dynamic Content Rendering

The Register component submits the form data to the backend and reports success or failure to the user:

```
import { useState } from "react";
import API from "../services/api";

function Register() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleRegister = async (e) => {
    e.preventDefault();

    try {
      const res = await API.post("/auth/register", {
        name,
        email,
        password,
      });

      alert(res.data.message);
    } catch (error) {
      alert(error.response?.data?.message || "Registration Failed");
    }
  };

  return (
    <div>
      <h2>Register</h2>
      <form onSubmit={handleRegister}>
        <input type="text" placeholder="Enter Name" value={name}
          onChange={(e) => setName(e.target.value)} />
        <input type="email" placeholder="Enter Email" value={email}
          onChange={(e) => setEmail(e.target.value)} />
        <input type="password" placeholder="Enter Password" value={password}
          onChange={(e) => setPassword(e.target.value)} />
        <button type="submit">Register</button>
      </form>
    </div>
  );
}
```

```
export default Register;
```

The Products component fetches and renders the list of temples/darshan products on mount:

```
import { useEffect, useState } from "react";
import API from "../services/api";

function Products() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetchProducts();
  }, []);

  const fetchProducts = async () => {
    try {
      const res = await API.get("/products");
      setProducts(res.data);
    } catch (error) {
      console.log(error);
    }
  };

  return (
    <div>
      <h2>Products</h2>
      {products.map((product) => (
        <div key={product._id} style={{ border: "1px solid black", margin: "10px", padding:
"10px" }}>
          <h3>{product.name}</h3>
          <p>{product.description}</p>
          <p>₹ {product.price}</p>
          <p>{product.category}</p>
        </div>
      ))}
    </div>
  );
}

export default Products;
```

Milestone 5: Deployment

In Milestone 5, the focus is on running Darshan Ease locally — both the backend and the frontend — and verifying the full booking flow end-to-end before final submission.

Activity 5.1: Preparing the Application for Local Deployment

- **Set Up the Backend:** Open a terminal, navigate into the server folder, and install dependencies:

```
cd server  
npm install
```

- **Configure Environment Variables:** Inside the server folder, create a .env file with your MongoDB connection string and port:

```
MONGO_URI=mongodb://localhost:27017/darshanease  
PORT=8000
```

- **Start the Backend Server:** Run the backend using nodemon:

```
nodemon index.js
```

The server should start on <http://localhost:8000>.

Activity 5.2: Local Testing and Verification

- **Set Up the Frontend:** Open a new terminal, navigate into the client folder, and install dependencies:

```
cd client  
npm install
```

- **Start the Frontend:** Start the React development server:

```
npm run dev
```

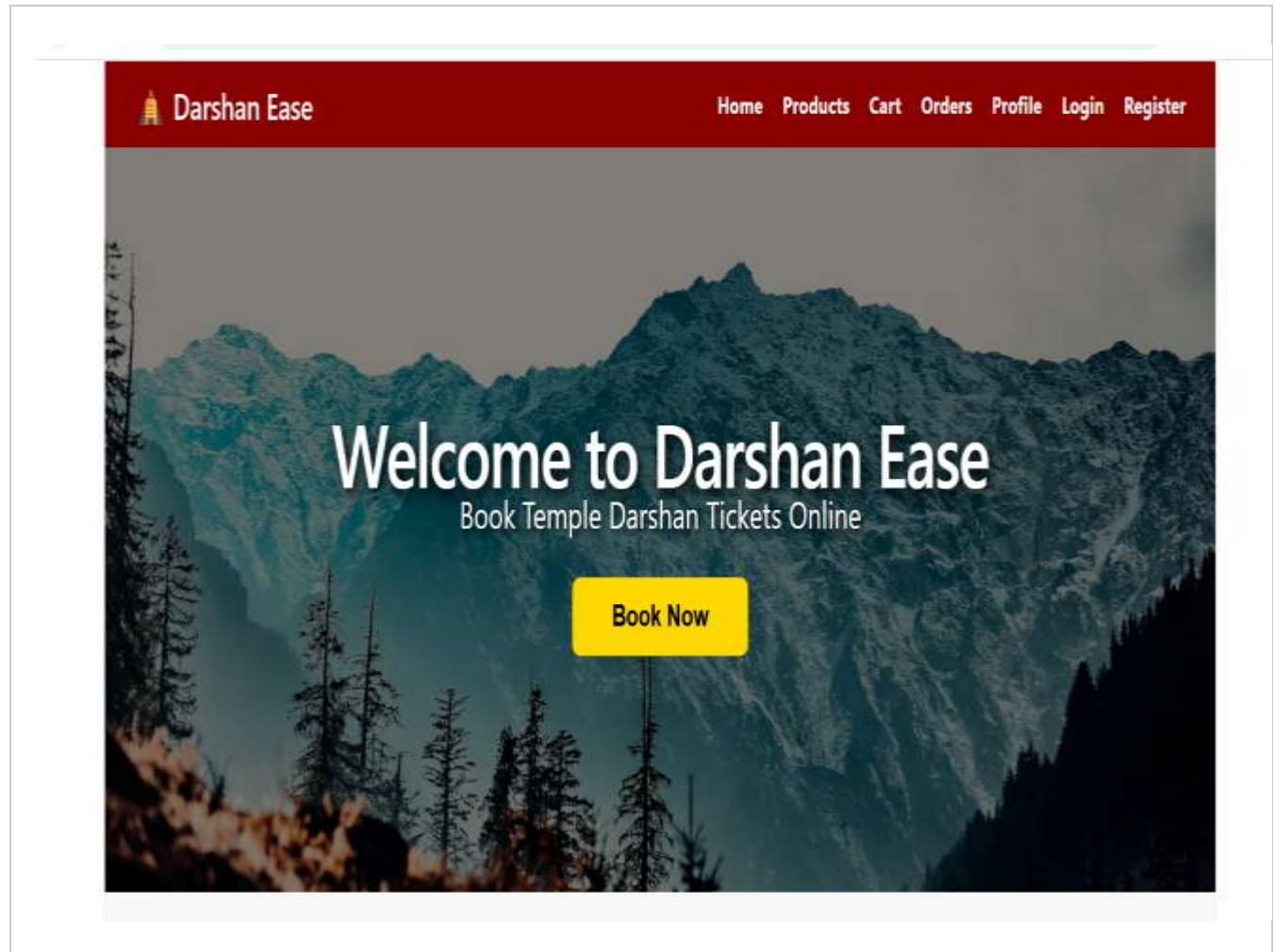
The app should now be running on <http://localhost:5173>.

- **Verify the Flow:** Register a new user, log in, browse temples/products, book a darshan slot, and confirm the booking appears in the order history — to verify the full flow works end-to-end.

Exploring the Website's Web Pages:

Home Page:

The landing page introduces Darshan Ease and gives devotees an overview of the temple darshan ticket booking service, with navigation to browse temples, register, or log in.



Register / Login Pages:

Simple forms where a new devotee creates an account with name, email, and password, or an existing user logs in with email and password to receive a JWT for authenticated access.

 Darshan Ease

HomeProductsCartOrdersProfileLoginRegister

Register

Enter Name

Enter Email

Enter Password

Register

Join Darshan Ease Today 

 Darshan Ease

HomeProductsCartOrdersProfileLoginRegister

Login

Enter Email

Enter Password

Login

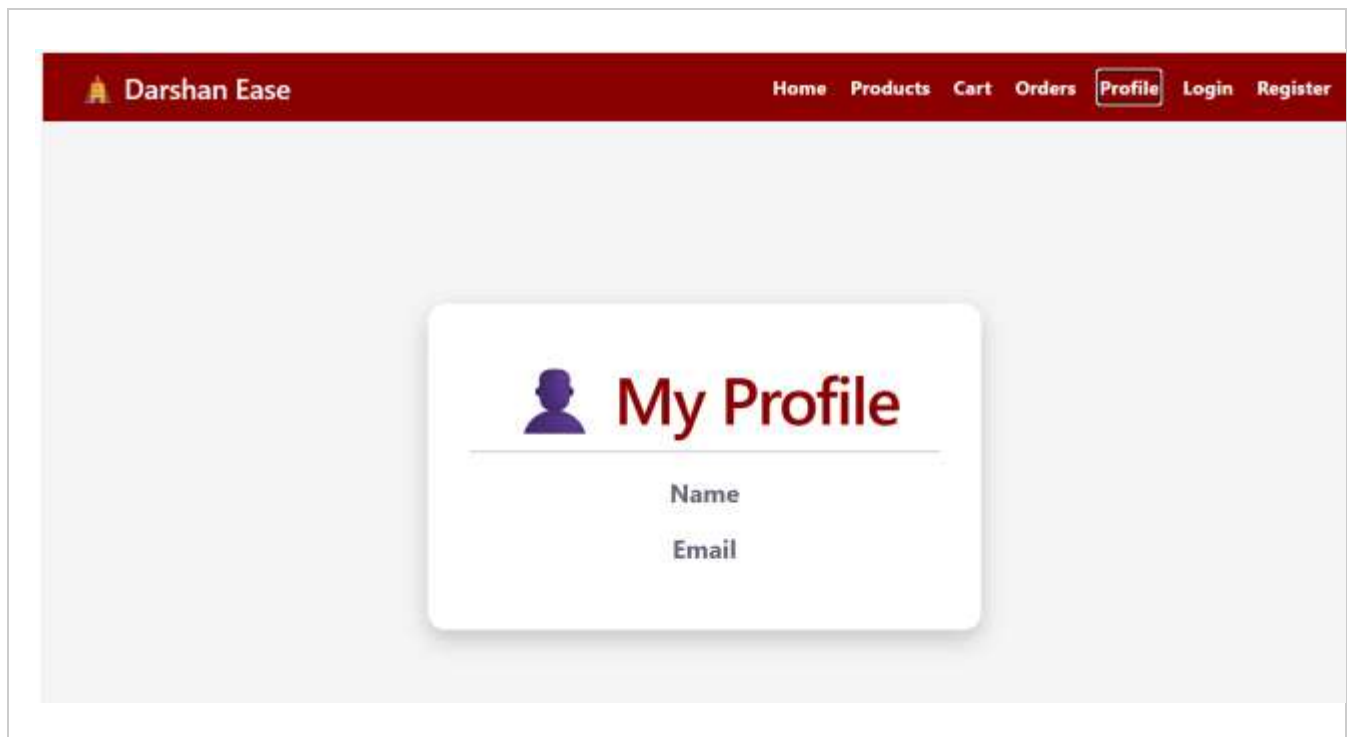
Products / Temples Page:

Displays the list of available temples/darshan products fetched from the backend, each showing its name, description, price, and category.



Profile Page:

Lets a logged-in user view and update their personal profile information.



Orders / Booking History Page:

Shows the user's current and past darshan bookings, including which temple/slot was booked and the booking status.



My Orders

VIP Darshan Ticket

User : Amrutha

Quantity : 2

Total : ₹ 1000

Status : **Pending**

VIP Darshan Ticket

User : Amrutha

Quantity : 2

Total : ₹ 1000

Status : **Pending**

Conclusion:

Darshan Ease is a full-stack web platform built with React, Node.js/Express, and MongoDB that streamlines temple darshan ticket booking. It follows the MVC architectural pattern on the backend, uses JWT-based authentication to secure every booking and profile action, and gives devotees a fast, reliable way to register, browse temples, and book darshan slots online instead of relying on offline queues or phone calls. The project, which included backend setup, core feature development, frontend design, and local deployment, highlights how a well-structured MERN-style application can meaningfully improve a real-world, high-friction process for devotees and temple administrators alike. Looking ahead, Darshan Ease has clear room to grow — integrating an online payment gateway, adding QR-code based e-tickets, sending email/SMS booking confirmations, and building out an admin analytics dashboard would all extend the platform from a booking tool into a more complete temple management solution.